

Hand-Computer Control Interface

ECS 174

Nathan Bao, Karim Shami, Shengjun Xu

Fall 2025

Abstract

We present a hand-gesture recognition interface for controlling standard mouse functions on computer systems using real-time hand pose keypoints. Our method trains a lightweight neural classifier on a custom dataset consisting of six static hand poses: thumbs up, thumbs down, point, fist, peace, and five. We evaluate the system using training curves, test accuracy, and per-class performance metrics. The final model achieves a test accuracy of 0.9459 on a 6-class custom dataset, indicating strong generalization across the selected gesture categories. These results highlight the potential of using compact landmark-based classifiers for intuitive human-computer interaction.

1 Introduction

Hand-gesture interfaces provide a natural and intuitive alternative to traditional mouse and keyboard input. With the advancement of efficient hand-tracking models such as MediaPipe Hands, it is now feasible to build real-time gesture control systems on consumer hardware. In this project, we develop a hand gesture control interface that detects hand landmarks using MediaPipe and classifies gestures using a custom neural network. Our goal for this project is (i) to achieve robust recognition on different hand gestures, (ii) to evaluate model performance on our own dataset, and (iii) to create a working prototype of controlling mouse inputs on local devices.

Our approach draws inspiration from the HaGRID hand-gesture dataset, which provides large-scale labeled hand pose data for gesture recognition tasks. Due to the dataset's size and download limitations, we instead opted to create our own dataset while referencing the methodologies and gesture taxonomy used in existing resources such as HaGRID and its associated GitHub implementations. These works guided the design of our gesture classes, data collection pipeline, and overall system.

2 Methods

At a high level, our hand-computer control interface operates by capturing live video from a webcam, extracting 21-point hand landmarks via MediaPipe Hands, feeding the flattened landmark vector into a neural gesture classifier, and executing the corresponding mouse input when a gesture is detected. Below, we describe each component of the pipeline in detail, including data collection, model architecture, training procedure, and runtime control logic.

2.1 Landmark Extraction

Our initial approach consisted of taking labeled hand pose images and training a convolutional neural network. Rather than training/classifying with images as is, we went with a more explicit approach by extracting the hand landmarks directly using Mediapipe, an open-source computer vision framework developed by Google. Feeding an image into Mediapipe gives us the absolute (x,y) positions (0 to 1) for all 21 landmarks on the hand (if detected). As shown below, each hand pose can be represented as a list of 21 (x,y) datapoints.

For consistency, we preprocess the data points by scaling to the input image's dimensions and ensuring all landmarks are relative to the wrist, which is at (0,0). Through this approach, hands are automatically detected by Mediapipe and the hand's palm and finger features are extracted as explicit data points that can be fed into our neural network model. Landmark extraction vastly reduces the learning overhead a model would otherwise have if fed images as is, which is great given our project is limited by time and compute.

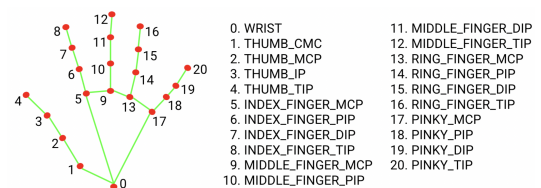


Figure 1: Hand's 21 landmarks from Mediapipe.

2.2 Data Collection

We store our labeled data in a csv file, where each data point is the label followed by the landmarks' 21 (x,y) positions. This means each row in our csv is 43 columns. Each label is a number 0 to 5 encoding the following 6 poses:

- 0: FIST
- 1: POINT
- 2: PEACE
- 3: THUMBS_UP
- 4: THUMBS_DOWN
- 5: FIVE

Given our classifier is fed very explicit data for a few gestures, we didn't need a very large dataset, which is why we built an interface to manually log data points for each label. This interface is just like our frontend hand-control system: it records video, and we use keystrokes to select the label and log the current video frame as a data point.

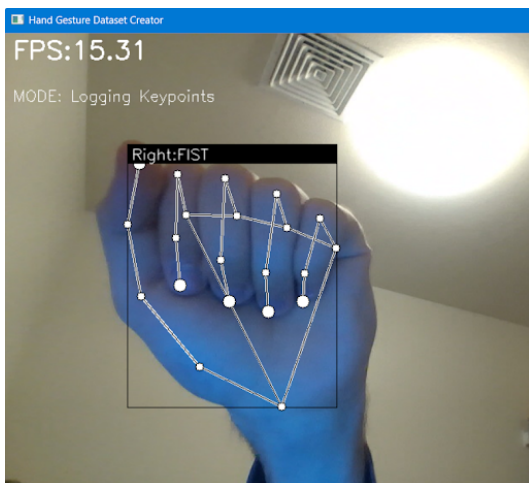


Figure 2: Logging hand poses

Given how we have direct control over the dataset, we made sure to apply implicit data augmentation by varying our hand poses, such as tilting, moving closer/farther to the camera, shifting the finger positions, and considering different ways of making a particular hand pose. For example, some people wrap their fingers around their thumb when making a fist, versus keeping the thumb to the side, or even placing the thumb in front of the fingers. Even if we missed a few variations, our model differentiates well since we picked noticeably distinct hand poses.

After logging, our dataset consists of 180 data points, 6 labels, and 30 data points per label.

2.3 Model Architecture & Training

Our model architecture is a multi-layer perceptron (MLP) classifier that takes as input 42 keypoint features and has

2 hidden layers. Our feature compression goes from 42 to 20, 20 to 10, then 10 to the number of classes, which in our case is 6 hand poses. As shown below, we use dropout regularization and ReLU activations.

Listing 1: Model Architecture

```
class KeyPointModel(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Dropout(0.2),
            nn.Linear(42, 20),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(20, 10),
            nn.ReLU(),
            nn.Linear(10, num_classes)
        )

    def forward(self, x):
        return self.net(x)
```

We use an 80/20 train/test split, training for 100 epochs with a batch size of 32 on Google Colab. We use the Adam optimizer, a learning rate of 0.001, and cross-entropy loss. After training, we're able to successfully identify hand poses based on the hand's landmarks.

2.4 Computer Controller

The last piece of the puzzle is the computer controller, which takes the identified action and executes it. We used PyAutoGUI, a GUI (graphical user interface) Python module that can execute OS-level input. This is great for basic actions like clicking and scrolling, but struggles in mouse movement, especially when a program/game requires raw input. We solve this using win32api, a Python module that can handle raw mouse input. One limitation is that this module only works on Windows, so raw input functionality is only available on Windows, and we default to PyAutoGUI for other platforms. We decided to map hand gestures to mouse actions, as shown below:

- FIST: Neutral, stop all actions
- POINT: Left-click down
- PEACE: Right-click down
- THUMBS_UP: Scroll up
- THUMBS_DOWN: Scroll down
- FIVE: Double left-click
- Hand position: Move cursor

Hand gestures are mutually exclusive, but can be executed at the same time as moving the cursor, enabling actions such as click and drag.

For cursor movement, we take the position at the center of the palm relative to the camera window, normalize it (0 to 1), and move the cursor accordingly. We implemented two movement modes: absolute and relative. In both cases we apply smoothing between the previous and next

positions.

Absolute mode moves the cursor to the screen position in proportion to the hand's position in the camera window. To account for issues reaching the edge of the screen, we mapped the camera range to a larger virtual range, applying a 1.5 expansion factor, then clamping. Since the human hand and Mediapipe's detected landmarks cause tiny jitters in the hand's position, we apply a small deadzone to reduce the small jittery movement.

Relative mode moves the cursor an incremental amount of distance based on the hand's relative position to the center of the camera window. There's a deadzone at the center to prevent unwanted cursor movement. Based on the (dy,dx) relative to the center, we scale that difference in proportion to a sensitivity variable.

Given all this, our computer controller enables us to execute all basic functions of the mouse.

2.5 Control Interface

Our frontend control interface puts everything together, where we use OpenCV, a widely used computer vision library, to capture video and display it in a resizable overlay window. Frame by frame, we do landmark extraction, call the trained pose predictor, and execute the action based on hand pose and palm position. Users can switch between absolute and relative cursor movement with alt+R and quit the control interface program with alt+Q. We draw the hand landmarks and predicted hand pose label in the camera window to provide live feedback.

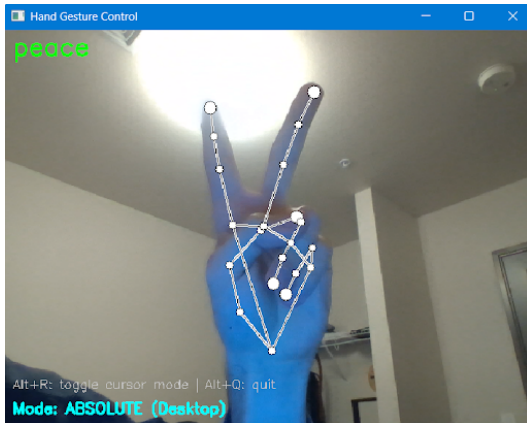


Figure 3: Control Interface Window

3 Results

3.1 Training Results

As we can see from the charts after, the model converges smoothly over 100 epochs. The training accuracy increasing steadily to 0.8750 and loss decreasing steadily to 0.4222. Pattern suggests that the optimizer is making stable progress without sudden jumps or divergence.



Figure 4: Training Loss

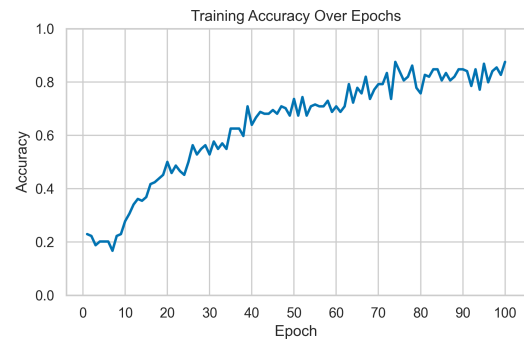


Figure 5: Training Accuracy

Epoch 1/100	Loss: 1.7784	Acc: 0.2292
Epoch 2/100	Loss: 1.7771	Acc: 0.2222
Epoch 3/100	Loss: 1.7701	Acc: 0.1875
Epoch 4/100	Loss: 1.7626	Acc: 0.2014
Epoch 5/100	Loss: 1.7505	Acc: 0.2014
Epoch 6/100	Loss: 1.7447	Acc: 0.2014
Epoch 7/100	Loss: 1.7314	Acc: 0.1667
Epoch 8/100	Loss: 1.7285	Acc: 0.2222
Epoch 9/100	Loss: 1.7300	Acc: 0.2292
Epoch 10/100	Loss: 1.6955	Acc: 0.2778
Epoch 11/100	Loss: 1.6733	Acc: 0.3056
Epoch 12/100	Loss: 1.6627	Acc: 0.3403
Epoch 13/100	Loss: 1.6312	Acc: 0.3611
Epoch 14/100	Loss: 1.6124	Acc: 0.3542
Epoch 15/100	Loss: 1.5790	Acc: 0.3681
Epoch 16/100	Loss: 1.5654	Acc: 0.4167
Epoch 17/100	Loss: 1.5323	Acc: 0.4236
Epoch 18/100	Loss: 1.5276	Acc: 0.4375
Epoch 19/100	Loss: 1.4644	Acc: 0.4514
Epoch 20/100	Loss: 1.4342	Acc: 0.5000
Epoch 21/100	Loss: 1.4492	Acc: 0.4583
Epoch 22/100	Loss: 1.3624	Acc: 0.4861
Epoch 23/100	Loss: 1.3641	Acc: 0.4653
Epoch 24/100	Loss: 1.3476	Acc: 0.4514
...		
Epoch 97/100	Loss: 0.4686	Acc: 0.8403
Epoch 98/100	Loss: 0.5069	Acc: 0.8542
Epoch 99/100	Loss: 0.4606	Acc: 0.8264
Epoch 100/100	Loss: 0.4222	Acc: 0.8750

Figure 6: Loss + Accuracy Per Epoch

3.2 Test Accuracy

On the held-out test set, the model achieved a final accuracy of 0.9459. The model correctly recognized the majority of previously unseen gesture samples.

Test Accuracy: 0.9459

Figure 7: Test Accuracy

3.3 Confusion Matrix Analysis

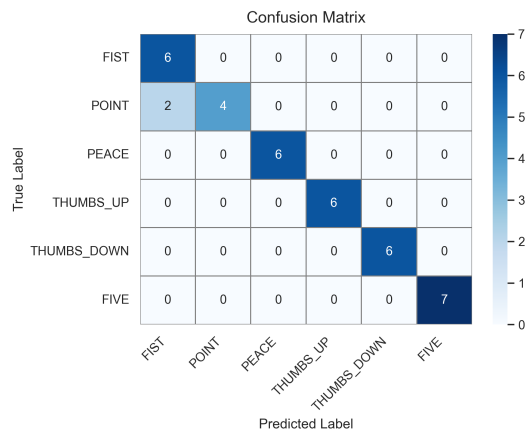


Figure 8: Confusion Matrix

The confusion matrix (Fig. 8) shows strong performance on gestures such as FIST, PEACE, THUMBS_UP, THUMBS_DOWN, and FIVE, with confusion occurring primarily on gestures such as POINT. These misclassifications likely arise because the keypoints for these gestures are visually similar in finger orientation.

3.4 Per-Class Accuracy

Per-class results show that the classifier performs best on gestures with distinct spatial keypoint patterns such as FIST, PEACE, THUMBS_UP, THUMBS_DOWN, and FIVE. These plots reinforce the confusion-matrix findings by highlighting which gestures the model reliably distinguishes and which (primarily POINT) are more prone to error.

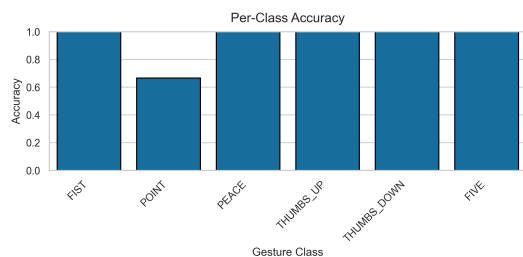


Figure 9: Per-Class Accuracy

FIST	: 1.000
POINT	: 0.667
PEACE	: 1.000
THUMBS_UP	: 1.000
THUMBS_DOWN	: 1.000
FIVE	: 1.000

Figure 10: Explicit Accuracies of Each Gesture

4 Discussion

4.1 Use Cases

Given that our hand-computer control interface simulates a mouse, there are practically infinite use cases. In our tests, we used it for browsing Google, drawing in a paint program, playing chess, navigating the browser, and more.



Figure 11: Browsing and scrolling

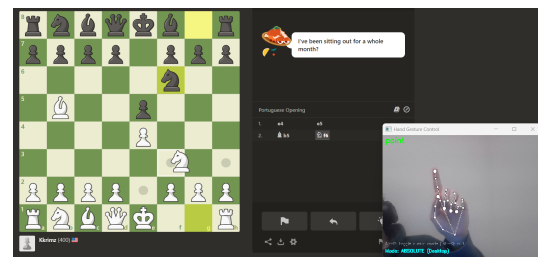


Figure 12: Playing chess, moving knight

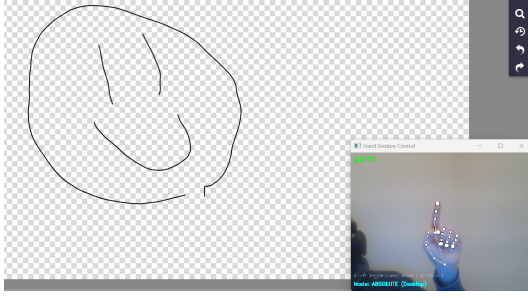


Figure 13: Drawing, holding left-click

One complex application we used our interface for is playing a game like Minecraft. We utilized our control interface to move the camera using relative movement, while using absolute movement when navigating UIs. In this example, we found it great to combine controls, having the left hand on the keyboard to control player movement, while the right hand controlled the mouse through our interface program.



Figure 14: Playing Minecraft, moving camera

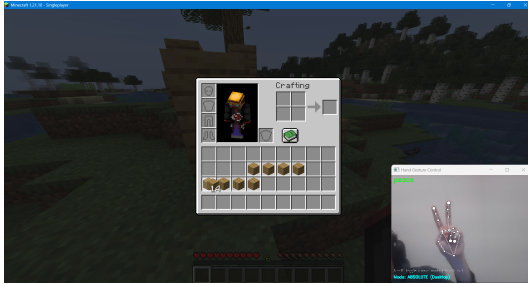


Figure 15: Playing Minecraft, dragging right-click in UI

4.2 Limitations

The main limitations of our current project are the size and limited diversity of our dataset. Compared to public datasets such as HaGRID, our dataset contains relatively few samples per gesture class, as all data was collected manually using keystrokes to label each captured frame. Another limitation is the model's ability to generalize to variations in lighting, background clutter, camera angles, and different users with diverse hand shapes, given that our dataset was collected in a controlled indoor environment.

4.3 Future Work

There are several promising directions for extending our current project. First, expanding the size and diversity of our dataset would improve the robustness of our model. Second, incorporating multi-hand input rather than single-hand input would unlock interactions similar to those on the Apple Vision Pro, enabling more advanced gesture control. Third, we can incorporate custom mappings for different preset hand gestures in application-specific contexts, such as video games like Minecraft for mining, walking, jumping, item selection, and building. In its current form, our system is primarily suited for everyday browser use, with one hand on the keyboard and the other performing gestures, rather than fully replacing all input in complex applications or games.

5 Conclusion

In this project, we developed a lightweight gesture-based computer control interface using MediaPipe keypoint extraction and a custom neural classifier. Our model successfully learned to recognize six different hand gestures from 21-point landmark inputs and achieved a reasonable 94.59% test accuracy on a limited, custom dataset. Through training and evaluation, we demonstrated that compact neural architectures can interpret hand gestures effectively in real-time. Overall, despite some limitations, our project provides a solid foundation for more advanced human-computer interaction applications and opens doors to future extensions that can better adapt to different users and usage scenarios.

References

- [1] Google Research. “MediaPipe Hands: Real-Time Hand Tracking.”
https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- [2] Zhang, F., Bazarevsky, V., Vakunov, A., et al. “MediaPipe Hands: On-Device Real-Time Hand Tracking.” Workshop on Computer Vision for AR/VR, CVPR (2020).
<https://arxiv.org/abs/2006.10214>
- [3] Daniel Bourke. “PyTorch for Deep Learning — 02. PyTorch Classification.”
https://www.learnpytorch.io/02_pytorch_classification/
- [4] Hukenov, V. “HaGRID — HAnd Gesture Recognition Image Dataset (GitHub Repository).”
<https://github.com/hukenovs/hagrid>
- [5] Kapitanov, A. “HaGRID Hand Gesture Dataset.” Kaggle.
<https://www.kaggle.com/datasets/kapitanov/hagrid/data>
- [6] Kapitanov, A., Hukenov, V. “HaGRID: HAnd Gesture Recognition Image Dataset.” CVPR Workshops (2022).
<https://arxiv.org/abs/2206.08219>
- [7] Molchanov, P., Yang, X., Gupta, S., Kim, K. “Hand Gesture Recognition: A Survey.” IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 2015.
- [8] Kim, M., Lee, D., Kim, H., Ro, Y. “Deep Learning-Based Hand Pose Estimation: A Review.” Pattern Recognition (2021).
- [9] Yama, K., Sakamoto, Y. “Real-Time Hand Gesture Recognition Using Landmarks and Lightweight Neural Models.” Procedia Computer Science (2018).
- [10] Wang, Y., Liu, J. “Natural User Interfaces: Trend and Evolution.” IEEE Computer (2012).
- [11] Srivastava, N., Hinton, G., Krizhevsky, A., et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting.” Journal of Machine Learning Research (JMLR), 2014.
- [12] Kingma, D., Ba, J. “Adam: A Method for Stochastic Optimization.” arXiv preprint arXiv:1412.6980 (2014).
<https://arxiv.org/abs/1412.6980>

Author Contributions

Nathan Bao — Wrote the Abstract, Introduction, Related Works, Results, Discussion, and Conclusion sections of the report. Developed and updated the training notebook to incorporate an 80/20 train/test split, additional evaluation graphs, and metrics for both training and testing. Implemented the computer controller logic using PyAutoGUI and platform-specific input, connecting it to the gesture classifier with smoothing and deadzoning for polished cursor movement. Refined hand-position handling and added persistent left-click behavior when the POINT gesture is held to support click-and-drag interactions.

Karim Shami — Developed the dataset logger and hand landmark extraction pipeline, including the logic to record labeled samples from the live camera feed into a CSV format. Implemented the MLP-based gesture classification model and integrated it with the landmark features. Built the frontend control interface, ensuring a resizable overlay window and support for relative mouse movement suitable for games. Collected and labeled the custom hand-gesture dataset used for training, applying varied poses and viewpoints to improve coverage. For the report, primarily wrote and refined the Methods section describing data collection, model architecture, and the runtime control pipeline.

Shengjun Xu — Complemented the report by expanding the Dataset and resulting section, adding the relevant figures and writing explanations of training procedure. For project, extended the system to support FIVE gesture, collected and labeled new samples with our data-logging interface.